



汕头大学 · 融合集成智能感知课题组

使用范围

内部使用

电磁波墙体探测入门

用电磁波探测墙内导线分布——从原理到实验



初稿撰写	jwhu
版本号	V1.0
发布日期	2026 年 6 月

目录

1	为什么要用电磁波探测墙体导线？	3
1.1	问题背景	3
1.2	电磁方法分类	3
1.3	整体技术路线	3
2	电磁波探测基础理论	4
2.1	电磁波在介质中的传播	4
2.2	反射与透射	4
2.3	距离分辨率	4
2.4	穿透深度与频率选择	5
3	GPR 信号模型与仿真	5
3.1	发射信号模型	5
3.2	正演仿真：单根导线的 A-scan	6
3.3	多根导线的 B-scan 仿真	8
4	SAR 成像算法	9
4.1	为什么需要 SAR 成像？	9
4.2	后向投影（Back-Projection）算法	9
4.3	Migrated 图像 vs 原始 B-scan	11
5	硬件方案参考	11
5.1	方案一：软件定义雷达（SDR）	11
5.2	方案二：超宽带脉冲雷达（定制）	11
5.3	方案三：商用探地雷达	11
5.4	数据采集流程	12
6	信号处理流水线	12
7	实验设计与结果验证	14
7.1	缩比实验方案	14
7.2	与数字孪生数据联动	15
7.3	评估指标	16
8	进阶方向	16
8.1	算法提升	16
8.2	硬件扩展	16
8.3	推荐学习路线	16

1 为什么要用电磁波探测墙体导线？

1.1 问题背景

墙内导线探测在以下场景中具有实际需求：

- **建筑改造**：打孔、开槽前需确认导线位置，避免切断线路
- **无损检测**：老旧建筑管线普查，不破坏墙体
- **安全排查**：检测隐蔽布线是否符合规范
- **逆向建模**：为 BIM（建筑信息模型）提供隐蔽管线数据

传统方法——敲击听音、红外热成像（通电后发热）、X 射线——各有局限。**电磁波方法**依靠金属导线对电磁波的反射/散射响应实现非接触、非破坏、实时成像。

1.2 电磁方法分类

方法	原理	探测深度	分辨率
探地雷达（GPR）	脉冲电磁波反射	0.1–2 m	厘米级
电磁感应（EMI）	涡流效应	0–0.3 m	分米级
电容耦合探测	介电常数差异	0–0.1 m	厘米级
低频磁探测	磁场梯度	0–0.5 m	分米级

本教程聚焦探地雷达（GPR）方法：原理较直观，成像效果好，是工程中最常用的方法。自制的超宽带雷达配合 SAR 成像算法即可实现墙面导线的二维成像。

1.3 整体技术路线

1. 理解电磁波与材料相互作用——反射、衰减、极化
2. 设计或选用天线——中心频率 1–4 GHz 的宽带天线
3. 搭建测量平台——天线沿墙面扫描，采集 A-scan 数据
4. 信号处理——去直达波、背景去除、增益补偿
5. SAR 成像——合成孔径聚焦，重建导线位置的 B-scan/C-scan 图像
6. 与数字孪生数据联动——将成像结果与 BIM 模型或 Blender 重建对比

2 电磁波探测基础理论

2.1 电磁波在介质中的传播

电磁波在非磁性介质中的传播速度取决于介质的相对介电常数 ϵ_r :

$$v = \frac{c}{\sqrt{\epsilon_r}}$$

对于典型墙体材料:

材料	ϵ_r	v (m/ns)	衰减 (dB/m @ 1 GHz)
空气	1.0	0.300	0
干砖墙	4-6	0.12-0.15	5-15
混凝土	6-12	0.09-0.12	10-50
石膏板	2.5-3.5	0.16-0.19	3-8
金属 (导线)	∞ (导体)	全反射	N/A

2.2 反射与透射

电磁波在两种介质界面上的反射系数由 Fresnel 公式给出 (垂直入射):

$$\Gamma = \frac{\sqrt{\epsilon_{r1}} - \sqrt{\epsilon_{r2}}}{\sqrt{\epsilon_{r1}} + \sqrt{\epsilon_{r2}}}$$

金属导线的 $\epsilon_{r2} \rightarrow \infty$, 因此 $\Gamma \approx 1$ ——几乎所有能量都被反射回来。这正是电磁波能有效探测导线的物理基础。

2.3 距离分辨率

距离分辨率取决于发射脉冲的带宽:

$$\Delta R = \frac{c}{2B\sqrt{\epsilon_r}}$$

其中 B 是信号带宽。对于 $B = 2 \text{ GHz}$ 的超宽带信号和 $\epsilon_r = 6$ (砖墙):

```
1 c = 3e8 # 光速 (m/s)
2 B = 2e9 # 带宽 2 GHz
3 eps_r = 6.0 # 砖墙介电常数
4
5 delta_R = c / (2 * B * np.sqrt(eps_r))
6 print(f"空间分辨率: {delta_R*100:.1f} cm")
```

```
7 # 输出：空间分辨率：3.1 cm
```

这意味着间隔 3 cm 以上的两根导线可以区分开。

2.4 穿透深度与频率选择

```
1 import numpy as np
2
3 def penetration_depth(f, eps_r=6.0, loss_tan=0.1):
4     """
5     估算电磁波在介质中的穿透深度（趋肤深度）
6     f: 频率 (Hz)
7     loss_tan: 损耗角正切
8     """
9     c = 3e8
10    alpha = np.pi * f * np.sqrt(eps_r) * loss_tan / c
11    skin_depth = 1 / alpha # 振幅衰减到 1/e 的距离
12    return skin_depth
13
14 for f_mhz in [500, 1000, 2000, 3000]:
15     sd = penetration_depth(f_mhz * 1e6)
16     print(f"{f_mhz:4d} MHz | 穿透深度: {sd:.2f} m")
```

频率选择的权衡：

- 低频（500 MHz）：穿透深（ $\gg 1$ m），但分辨率低（10 cm）
- 高频（3 GHz）：分辨率高（2 cm），但穿透浅（0.2 m）
- 折中：1–2 GHz 适合典型墙体（15–30 cm 厚）的导线探测

3 GPR 信号模型与仿真

3.1 发射信号模型

常用超宽带脉冲：Ricker 子波（一阶高斯导数）：

$$s(t) = (1 - 2(\pi f_c t)^2) e^{-(\pi f_c t)^2}$$

其中 f_c 是中心频率。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def ricker_wavelet(fc, dt, n_samples=256):
5     """生成 Ricker 子波（一阶高斯导数）"""
6     t = np.arange(n_samples) * dt - n_samples * dt / 2
7     tau = np.pi * fc * t
8     w = (1 - 2 * tau**2) * np.exp(-tau**2)
9     return t, w
10
11 fc = 1.5e9 # 1.5 GHz
12 dt = 1 / (6e9) # 采样间隔
13 t, w = ricker_wavelet(fc, dt)
14
15 plt.figure(figsize=(10, 3))
16 plt.plot(t*1e9, w)
17 plt.xlabel("时间 (ns)"); plt.ylabel("幅度")
18 plt.title(f"Ricker 子波 @ {fc/1e9:.1f} GHz, 带宽 ~{fc/1e9:.1f} GHz")
19 plt.grid(alpha=0.3)
20 plt.tight_layout()
21 plt.savefig("ricker_wavelet.png", dpi=150)
```

3.2 正演仿真：单根导线的 A-scan

对墙内的一根导线，雷达接收到的信号包括：

1. 直达波（Direct Coupling）：发射天线直接耦合到接收天线
2. 墙面反射：空气-墙体界面的反射
3. 目标反射：导线产生的强反射（我们的目标）

```
1 def simulate_a_scan(wavelet, dt, wall_thickness=0.2, wire_depth=0.12,
2                     eps_wall=6.0, wire_radius=0.005):
3     """
4     模拟墙内单根导线的 A-scan 信号
5
6     参数：
7         wavelet: Ricker 子波数组
8         dt: 时间步长 (s)
```

```
9     wall_thickness: 墙体厚度 (m)
10     wire_depth: 导线深度 (从墙面算起, m)
11     eps_wall: 墙体介电常数
12
13     返回:
14         t: 时间轴 (s)
15         signal: 接收信号
16     """
17     c = 3e8
18     v = c / np.sqrt(eps_wall)
19     n = len(wavelet)
20     signal = np.zeros(n)
21
22     # 1. 直达波 (天线耦合)
23     signal += 0.3 * np.roll(wavelet, 2) # 小延时
24
25     # 2. 墙面反射 (空气-墙体界面, 深度 0)
26     gamma_front = (1 - np.sqrt(eps_wall)) / (1 + np.sqrt(eps_wall))
27     t_front = 2 * 0.01 / c # 天线距墙面 1cm
28     delay_front = int(t_front / dt)
29     signal += gamma_front * np.roll(wavelet, delay_front)
30
31     # 3. 导线反射
32     t_wire = 2 * (0.01 + wire_depth) / v # 双程: 空气+墙体
33     delay_wire = int(t_wire / dt)
34
35     # 导线反射系数 (近似为金属柱体)
36     gamma_wire = 0.8 * (wire_radius / 0.01) # 半径越粗越强
37     gamma_wire = min(gamma_wire, 1.0)
38     wire_signal = gamma_wire * wavelet
39     # 墙体传播衰减
40     atten = np.exp(-2 * wire_depth * 10) # 简单衰减模型
41     signal += atten * np.roll(wire_signal, delay_wire)
42
43     # 加噪声
44     signal += np.random.randn(n) * 0.02
45
46     t_axis = np.arange(n) * dt
47     return t_axis, signal
```

3.3 多根导线的 B-scan 仿真

B-scan 是天线沿墙面移动时采集的一系列 A-scan 组成的 2D 图像（横轴 = 天线位置，纵轴 = 时间/深度）：

```
1 def simulate_b_scan(n_positions=100, wire_config=None):
2     """
3     模拟 B-scan 数据
4
5     wire_config: [(depth, x_position), ...]
6     例: [(0.10, 0.3), (0.12, 0.6), (0.08, 0.8)]
7     """
8     fc, dt = 1.5e9, 1/6e9
9     t, w = ricker_wavelet(fc, dt)
10
11     if wire_config is None:
12         wire_config = [(0.10, 0.3), (0.12, 0.6)]
13
14     positions = np.linspace(0, 1, n_positions)
15     bscan = np.zeros((n_positions, len(w)))
16
17     for i, x in enumerate(positions):
18         # 所有导线对此位置的贡献
19         combined = np.zeros(len(w))
20         for depth, wx in wire_config:
21             # 导线与天线的水平距离
22             dx = abs(x - wx)
23             if dx > 0.15: # 超出天线波束范围
24                 continue
25             # 视深度（考虑斜距）
26             apparent_depth = np.sqrt(depth**2 + dx**2)
27             _, a_scan = simulate_a_scan(w, dt, wire_depth=apparent_depth)
28             # 距离越远散射越弱
29             a_scan *= np.exp(-dx * 20)
30             combined += a_scan
31         bscan[i] = combined
32
33     return positions, t, bscan
34
35
36 # 仿真并显示 B-scan
```



```

37 positions, t, bscan = simulate_b_scan(100, [(0.10, 0.3), (0.12, 0.6),
      (0.08, 0.8)])
38 depth = t * 3e8 / np.sqrt(6.0) * 100 # 转换为 cm
39
40 plt.figure(figsize=(10, 6))
41 plt.imshow(bscan.T, aspect='auto', cmap='gray',
42           extent=[positions.min(), positions.max(), depth.max(), depth.
      min()])
43 plt.xlabel("天线位置 (m)"); plt.ylabel("深度 (cm)")
44 plt.title("模拟 B-scan: 三根导线")
45 plt.colorbar(label="幅度")
46 plt.tight_layout()
47 plt.savefig("simulated_bscan.png", dpi=150)

```

B-scan 中，每根导线呈现为一条双曲线（因为天线在不同位置到导线的斜距不同，导致时间延迟变化）。双曲线的顶点就是导线的真实位置和深度。

4 SAR 成像算法

4.1 为什么需要 SAR 成像？

原始 B-scan 中导线呈双曲线形态，不利于直接判读。合成孔径雷达（SAR）成像将双曲线聚焦为点目标，形成更直观的墙体截面图像。

4.2 后向投影（Back-Projection）算法

最直观的 SAR 聚焦算法。对图像中的每个像素点，计算天线在各位置到该点的理论延时，提取对应时刻的 A-scan 值并叠加：

$$I(x, z) = \sum_{n=1}^N s_n \left(\frac{2\sqrt{(x - x_n)^2 + z^2}}{v} \right) \quad (1)$$

其中 $s_n(t)$ 是第 n 个天线位置的 A-scan， v 是墙中的电磁波速度。

```

1 def back_projection(bscan, positions, t, v, dx=0.01, dz=0.01,
2                   x_range=(-0.1, 1.1), z_range=(0, 0.25)):
3     """
4     后向投影 SAR 成像
5
6     参数:
7         bscan: (n_positions, n_samples) B-scan 数据
8         positions: 天线位置数组 (n_positions,)

```

```

9         t: 时间轴 ( $n\_samples$ ,)
10        v: 墙中电磁波速度 (m/s)
11        dx, dz: 图像分辨率 (m)
12        x_range: 成像水平范围
13        z_range: 成像深度范围
14        """
15        # 创建图像网格
16        xs = np.arange(x_range[0], x_range[1] + dx, dx)
17        zs = np.arange(z_range[0], z_range[1] + dz, dz)
18        image = np.zeros((len(zs), len(xs)))
19
20        # 对每个像素点
21        for ix, x_pix in enumerate(xs):
22            for iz, z_pix in enumerate(zs):
23                # 对所有天线位置
24                for ip, x_ant in enumerate(positions):
25                    r = 2 * np.sqrt((x_pix - x_ant)**2 + z_pix**2)
26                    t_delay = r / v
27                    # 线性插值提取对应时刻的幅度
28                    idx = t_delay / (t[1] - t[0])
29                    if 0 <= idx < len(t) - 1:
30                        i0, i1 = int(idx), int(idx) + 1
31                        frac = idx - i0
32                        amp = (1 - frac) * bscan[ip, i0] + frac * bscan[ip, i1]
33                        image[iz, ix] += amp
34
35        return xs, zs, image
36
37        # SAR 成像
38        v = 3e8 / np.sqrt(6.0)
39        xs, zs, sar_image = back_projection(bscan, positions, t, v)
40
41        plt.figure(figsize=(10, 6))
42        plt.imshow(sar_image, aspect='auto', cmap='hot',
43                  extent=[xs.min(), xs.max(), zs.max()*100, zs.min()*100])
44        plt.xlabel("水平位置 (m)"); plt.ylabel("深度 (cm)")
45        plt.title("SAR 聚焦成像: 三根导线清晰可见")
46        plt.colorbar(label="聚焦幅度")
47        plt.tight_layout()
48        plt.savefig("sar_image.png", dpi=150)

```

4.3 Migrated 图像 vs 原始 B-scan

原始 B-scan 中的双曲线在 SAR 聚焦后被压缩为亮斑，导线的真实位置（横坐标和深度）一目了然。

5 硬件方案参考

5.1 方案一：软件定义雷达（SDR）

最灵活的入门方案，使用 USRP / HackRF / ADALM-PLUTO 等 SDR 平台：

部件	推荐型号	备注
SDR 平台	HackRF One / PlutoSDR	HackRF 带宽 20 MHz (不够); PlutoSDR 带宽 56 MHz (可做 步进频)
天线	Vivaldi 天线 / 喇叭天线	1–6 GHz 超宽带
功放	Mini-Circuits ZX60 系列	可选，增加发射功率
扫描机构	步进电机 + 直线导轨	沿墙面自动扫描

步进频连续波（SFCW）方案：SDR 逐频率发射连续波，合成超宽带信号。这是低成本实现高频分辨率的方法。

5.2 方案二：超宽带脉冲雷达（定制）

- 脉冲发生器：阶跃恢复二极管（SRD）或 avalanche 晶体管产生亚 ns 脉冲
- 接收机：等效时间采样（equivalent-time sampling）——用低速 ADC 采样重复波形
- 带宽：0.5–4 GHz，理论分辨率可达 $< 3\text{ cm}$

5.3 方案三：商用探地雷达

不建议学生从零搭建硬件（周期长、调试复杂）。推荐先用商用设备采集数据，专注信号处理和成像算法研究：

- MALA ProEx / CX：1.6 GHz 天线，适合墙体探测
- GSSI SIR 4000：2.6 GHz 天线，分辨率极高

5.4 数据采集流程

1. 校准：在已知位置放置金属杆，测量真实深度 vs 雷达时间以标定 ϵ_r
2. 网格扫描：在墙面上画网格线，逐行扫描构建 3D 数据体
3. 标记起始点：每次扫描记录起始位置，后续拼接
4. 多次叠加：每个位置采集多次，平均以压制噪声

6 信号处理流水线

```
1 import numpy as np
2 import scipy.signal as signal
3 import matplotlib.pyplot as plt
4
5 def gpr_processing_pipeline(bscan, dt, method='standard'):
6     """
7     GPR 信号处理流水线
8
9     步骤：
10     1. 去直流 (Dewow) ——消除低频漂移
11     2. 时变增益 (TGC) ——补偿深部信号衰减
12     3. 带通滤波——去除噪声
13     4. 背景去除——消除水平连续界面（墙面反射）
14
15     参数：
16         bscan: (n_positions, n_samples)
17         dt: 采样时间间隔 (s)
18     """
19     n_pos, n_samp = bscan.shape
20
21     # 1. 去直流（每道减去均值）
22     bscan = bscan - bscan.mean(axis=1, keepdims=True)
23
24     # 2. 时变增益（指数增益补偿衰减）
25     t = np.arange(n_samp) * dt
26     gain = np.exp(t * 2e8) # 增益因子随深度增大
27     bscan = bscan * gain[np.newaxis, :]
28
29     # 3. 带通滤波
30     fs = 1 / dt
```

```
31     sos = signal.butter(4, [0.3e9, 3e9], btype='band', fs=fs, output='
        sos')
32     bscan_f = np.zeros_like(bscan)
33     for i in range(n_pos):
34         bscan_f[i] = signal.sosfilt(sos, bscan[i])
35
36     # 4. 背景去除 (减去所有 A-scan 的平均)
37     background = bscan_f.mean(axis=0, keepdims=True)
38     bscan_f = bscan_f - background
39
40     return bscan_f
41
42
43 def detect_wires(sar_image, threshold=0.5, min_distance=0.05):
44     """
45     从 SAR 聚焦图像中检测导线位置
46
47     参数:
48         sar_image: (nz, nx) SAR 聚焦图像
49         threshold: 相对峰值阈值
50         min_distance: 导线之间的最小距离 (m)
51
52     返回:
53         peaks_x: 导线水平位置 (m)
54         peaks_z: 导线深度 (m)
55     """
56     from scipy.ndimage import maximum_filter
57     from skimage.measure import label
58
59     # 归一化
60     img_norm = sar_image / sar_image.max()
61
62     # 局部最大值检测
63     neighborhood = int(min_distance / 0.01) # 假设像素间距 1cm
64     local_max = maximum_filter(img_norm, size=neighborhood)
65     maxima = (img_norm == local_max) & (img_norm > threshold)
66
67     # 连通域标记
68     labeled = label(maxima)
69     peaks = []
70     for region_id in range(1, labeled.max() + 1):
```

```
71     yi, xi = np.where(labeled == region_id)
72     if len(yi) == 0:
73         continue
74     # 取质心
75     cy, cx = np.mean(yi), np.mean(xi)
76     peaks.append((cx, cy))
77
78     if len(peaks) == 0:
79         print("未检测到导线")
80         return [], []
81
82     peaks = np.array(peaks)
83     print(f"检测到 {len(peaks)} 根导线:")
84     for i, (px, py) in enumerate(peaks):
85         print(f" 导线 {i+1}: 水平位置={px:.3f}m, 深度={py:.3f}m")
86
87     return peaks[:, 0], peaks[:, 1]
```

7 实验设计与结果验证

7.1 缩比实验方案

自制墙体模型：

1. 材料：砖块或加气砖 + 水泥砂浆砌筑一面 $1\text{ m} \times 1\text{ m}$ 的小墙
2. 预埋导线：在砌筑时预埋多根不同深度和走向的导线（单根 / 平行 / 交叉）
3. 标记真实位置：记录每根导线的精确 3D 坐标作为 ground truth
4. 墙面处理：如需真实感，可批刮腻子并刷漆

替代方案（实验室桌面实验）：

- 用沙箱或干砂代替墙体（均匀介质，便于验证算法）
- 在砂中不同深度埋设铜线 / 钢筋
- 用干燥木板 + 导线模拟墙体效果（介电常数 $\epsilon_r \approx 2-4$ ）

7.2 与数字孪生数据联动

如同之前的教程系列，仿真数据与实测数据形成互补循环：

```
1 def compare_simulation_vs_measurement(sim_bscan, real_bscan, positions,
2     depth):
3     """
4     对比仿真 B-scan 与实测 B-scan
5
6     用于验证正演模型的准确性和校正参数
7     """
8
9     from sklearn.metrics import mean_squared_error
10
11     # 对齐维度
12     min_pos = min(sim_bscan.shape[0], real_bscan.shape[0])
13     sim = sim_bscan[:min_pos]
14     real = real_bscan[:min_pos]
15
16     # 归一化
17     sim = sim / sim.max()
18     real = real / real.max()
19
20     # MSE
21     mse = mean_squared_error(sim.flatten(), real.flatten())
22     print(f"仿真 vs 实测 MSE: {mse:.6f}")
23
24     # 对比图
25     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
26     axes[0].imshow(sim.T, aspect='auto', cmap='gray',
27         extent=[positions.min(), positions.max(),
28             depth.max(), depth.min()])
29     axes[0].set_title("仿真 B-scan")
30     axes[1].imshow(real.T, aspect='auto', cmap='gray',
31         extent=[positions.min(), positions.max(),
32             depth.max(), depth.min()])
33     axes[1].set_title("实测 B-scan")
34     plt.tight_layout()
35     plt.savefig("sim_vs_real_bscan.png", dpi=150)
```

7.3 评估指标

指标	定义	典型值
定位误差	$\sqrt{\Delta x^2 + \Delta z^2}$	1–3 cm
深度估计误差	$ z_{\text{估}} - z_{\text{真}} $	0.5–2 cm
检出率	正确检出 / 总导线数	> 90%
信杂比	目标峰值 / 背景均值	> 10 dB

8 进阶方向

8.1 算法提升

1. **Migration 算法**：Kirchhoff 偏移、 f - k 偏移（Stolt 偏移）——比后向投影更高效
2. **全波形反演（FWI）**：迭代优化介质参数 $\varepsilon_r(x, z)$ 和 $\sigma(x, z)$ ，不仅能定位导线，还能重建壁内介电常数分布
3. **压缩感知**：利用导线在空间上是稀疏的这一先验，用更少的测量位置实现高分辨率成像
4. **深度学习**：训练 CNN 直接从 B-scan 映射到导线位置（跳过 SAR 聚焦步骤）

8.2 硬件扩展

- **多天线阵列**：使用 4–16 通道天线阵列，一次扫描覆盖更宽区域
- **极化测量**：改变天线极化方向（VV/HH/VH），利用导线对特定极化的强反射增强检测
- **连续波差分探测**：用相干探测测量墙体微小形变（== 电磁波干涉测量）

8.3 推荐学习路线

1. **第 1 周**：理解电磁波传播和反射理论，运行仿真代码生成 B-scan
2. **第 2 周**：实现 SAR 后向投影算法，从仿真数据重建导线图像
3. **第 3–4 周**：搭建实验室测试平台（SDR + 天线 + 步进扫描）
4. **第 5 周**：实际采集墙面数据，运行信号处理 pipeline
5. **第 6 周**：对比仿真 vs 实测，校正介质参数
6. **第 7–8 周**：尝试 f - k 偏移或深度学习端到端方法

附录 A：完整脚本索引

1. `waves_basics.py` —— 电磁波传播参数计算（速度、分辨率、穿透深度）
2. `ricker_wavelet.py` —— Ricker 子波生成
3. `a_scan_sim.py` —— 单根导线的 A-scan 仿真（含直达波/墙面/目标反射）
4. `b_scan_sim.py` —— 多根导线的 B-scan 仿真
5. `sar_imaging.py` —— 后向投影 SAR 聚焦成像
6. `gpr_processing.py` —— 完整信号处理 pipeline（去直流/增益/滤波/背景去除）
7. `wire_detection.py` —— 从 SAR 图像自动检测导线位置
8. `experiment_utils.py` —— 数据加载、标定、仿真-实测对比

附录 B：参考文献

1. Daniels D.J., *Ground Penetrating Radar*, 2nd ed., IET, 2004.
2. Jol H.M. (ed.), *Ground Penetrating Radar: Theory and Applications*, Elsevier, 2009.
3. Yilmaz O., *Seismic Data Analysis: Processing, Inversion, and Interpretation of Seismic Data*, SEG, 2001. (SAR Migration 算法经典参考)
4. Stolt R.H., "Migration by Fourier transform", *Geophysics*, 1978. (f - k 偏移原始论文)
5. Solimene R. et al., "GPR B-scan imaging by means of a compressive sensing approach", *IEEE GRSL*, 2011.
6. 汕头大学融合集成智能感知课题组, 《Blender 数字孪生实战》, 2026.
7. 汕头大学融合集成智能感知课题组, 《深度神经网络入门实战》, 2026.
8. 汕头大学融合集成智能感知课题组, 《麦克风阵列声学成像入门》, 2026.